

Software Requirements Specification for ESP32 Room Climate Station

Version 4

Prepared by Ihor Huz

Lodz University of Technology

May 22, 2026

Table of Contents

Software Requirements Specification	1
Revision History.....	3
1. Introduction.....	4
1.1. Purpose.....	4
The purpose of this Software Requirements Specification is to define the architectural, functional, and non-functional requirements for the ESP32 Room Climate Station. It establishes the system boundaries and serves as a formal blueprint for hardware and software development.....	
1.2. Document Conventions.....	4
This document adheres to the IEEE 830-1998 standard for Software Requirements Specifications. All requirements listed in Section 5 carry a "High" priority for implementation unless explicitly stated otherwise.	
1.3. Intended Audience and Reading Suggestions	4
This document is intended for software developers, hardware engineers, and project evaluators. Readers should begin with Section 2 for a high-level overview of the system architecture and actors before proceeding to the specific technical interfaces and requirements in Sections 4, 5, and 6.	
1.4. Product Scope	4
1.4.1. In Scope (Includes)	4
1.4.2. Out Scope (Excludes).....	4
1.5. References	5
2. Overall Description	5
2.1. Product Perspective.....	5
2.2. Product Features.....	5
2.3. User Classes and Characteristics.....	6
2.4. Operating Environment.....	6
2.5. Design and Implementation Constraints	6
2.6. User Documentation	6
2.7. Assumptions and Dependencies.....	7
3. System Features	7
3.1. Local Climate Monitoring and Display	7
3.2. Telegram Bot Integration	7
3.3. Web Dashboard and Device Management	7
3.4. Active Threshold Alerts.....	7
3.5. Dynamic Web Configuration	7
4. External Interface Requirements.....	7
4.1. User Interfaces Overview.....	7
4.2. Hardware Interfaces	7
4.3. Software Interfaces	7
5. System Features/Modules	8

5.1. Local Climate Monitoring and Display8

5.2. Telegram Bot Integration8

5.3. Web Dashboard and Device Management9

5.4. Active Threshold Alerts.....9

5.5. Dynamic Web Configuration9

6. Nonfunctional Requirements9

6.1. Performance9

6.2. Reliability.....9

6.3. Maintainability 11

Revision History

Name	Date	Reason for Changes	Version
Ihor Huz	April 27, 2026	Initial draft for Lab 1: Requirement Analysis	1
Ihor Huz	May 8, 2026	Added the UML Use Case Diagram. Updated Product Perspective, Product Features, and External Interface Requirements	2
Ihor Huz	May 15, 2026	Added UML Class Diagram (Figure 2). Synchronized feature numbering and descriptions between Section 3 and Section 5.	3

Ihor Huz	May 22, 2026	Added Telegram API Sequence Diagram (Figure 3), Firmware Lifecycle State Diagram (Figure 4), and SPIFFS Data Logging Activity Diagram (Figure 5) to document dynamic system behaviour.	4
----------	--------------	--	---

1. Introduction

1.1. Purpose

The purpose of this Software Requirements Specification is to define the architectural, functional, and non-functional requirements for the ESP32 Room Climate Station. It establishes the system boundaries and serves as a formal blueprint for hardware and software development.

1.2. Document Conventions

This document adheres to the IEEE 830-1998 standard for Software Requirements Specifications. All requirements listed in Section 5 carry a "High" priority for implementation unless explicitly stated otherwise.

1.3. Intended Audience and Reading Suggestions

This document is intended for software developers, hardware engineers, and project evaluators. Readers should begin with Section 2 for a high-level overview of the system architecture and actors before proceeding to the specific technical interfaces and requirements in Sections 4, 5, and 6.

1.4. Product Scope

The ESP32 Room Climate Station addresses this limitation by functioning as a centralized, persistently logging IoT environment monitor. It bridges the gap between local hardware and remote software by reading ambient conditions through a high-accuracy SHTC3 sensor and making that data available through three distinct interfaces: a local LCD, a web-based dashboard, and a Telegram bot. Furthermore, the system solves the common IoT problem of volatile memory loss by utilizing SPIFFS to store a 24-hour rolling buffer of environmental data. This ensures that in the event of a power failure, the user does not lose their historical trend data. The integration of mDNS and Over-The-Air (OTA) updates ensures the device remains a low-maintenance, headless appliance once deployed.

1.4.1. In Scope (Includes)

The following capabilities are explicitly included within the boundaries of this project:

- Continuous monitoring of ambient temperature and humidity via the SHTC3 sensor.
- Local data display via a hardware LCD and manual physical toggle button.
- Remote reporting and basic remote command execution via a Telegram Bot API.
- A local, mDNS-accessible web dashboard for visualizing 24-hour historical data trends.
- Persistent storage of the most recent 24 hours of data in the internal SPIFFS memory.
- Over-The-Air (OTA) firmware update capabilities via the local network.

1.4.2. Out Scope (Excludes)

The following capabilities are explicitly excluded from this system boundary to maintain project focus and hardware performance:

- Active Climate Control: The system acts strictly as an environmental monitor. It will not interface with or control external appliances (e.g., HVAC systems, humidifiers, or fans).
- Long-Term Cloud Storage: The system will not push data to external cloud databases (e.g., AWS, Firebase) for long-term retention beyond the rolling 24-hour local buffer.

- Complex Web Authentication: The local web dashboard assumes a trusted local network (climate.local) and will not feature multi-user login or complex role-based access control.
- Battery/Deep Sleep Optimization: Because the device must run a continuous web server and listen for Telegram commands, deep-sleep power saving states are excluded. The system assumes a continuous wall-power supply.

1.5. References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Espressif Systems. ESP32 Technical Reference Manual.
- Telegram Messenger LLP. Telegram Bot API Documentation.

2. Overall Description

2.1. Product Perspective

Maintaining optimal indoor climate conditions (temperature and humidity) is critical for personal health, equipment longevity, and general comfort. Traditional consumer thermometers provide only real-time snapshots of the environment, lacking historical context and remote accessibility. When users leave the room or building, they lose visibility into the environmental trends.

2.2. Product Features

The ESP32 Room Climate Station offers a wide range of environmental monitoring capabilities by balancing local hardware interactions with remote access. At its core, the system constantly monitors the local weather and displays real-time sensor data on a physical hardware screen. To assist remote users, it includes a Telegram Bot that handles periodic weather reports and conducts remote orders. The device in question also features a local web dashboard for visualising past data patterns and managing firmware through Over-The-Air (OTA) updates. Furthermore, the system encompasses proactive intelligence in the form of active threshold alerts, which send emergency notifications when conditions become dangerous, as well as a dynamic web configuration interface that allows users to securely update network and API credentials on the fly without reprogramming the device. The UML Use Case Diagram below shows how our primary actors (Home User and System Timer) and secondary actors (Telegram API) interact with these features across the system boundary:

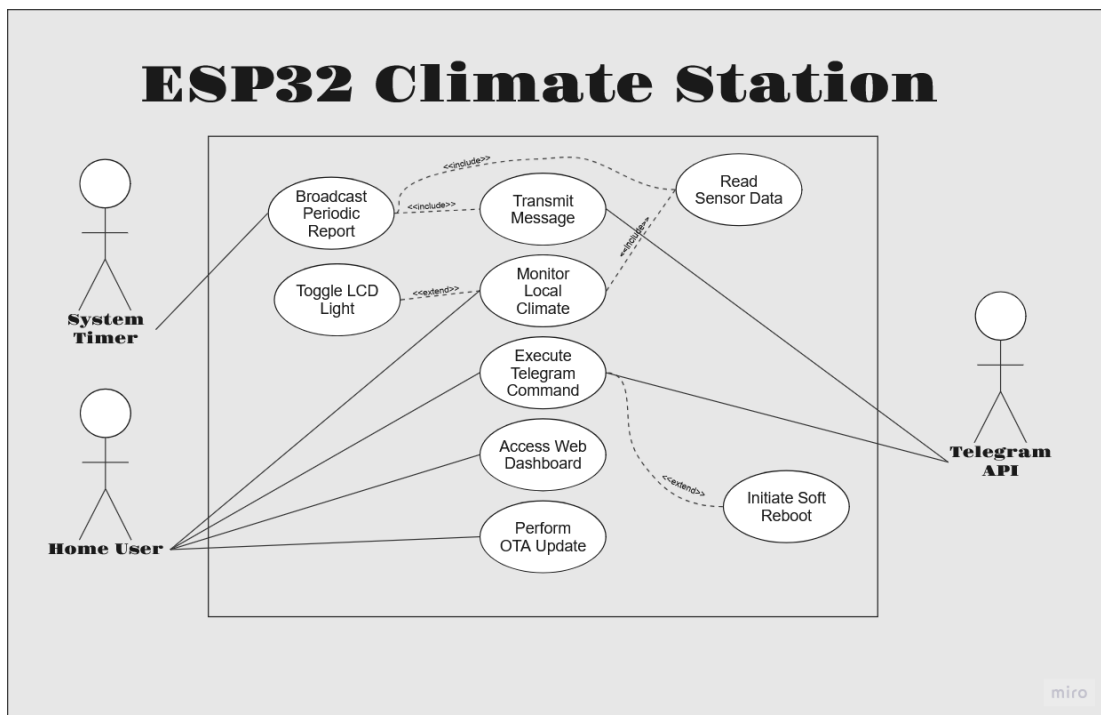


Figure 1: The UML Use Case Diagram for the Project.

The internal firmware architecture of the ESP32 is strictly object-oriented to ensure modularity and ease of maintenance. The system utilizes abstract parent classes to standardize hardware initialization, while strictly separating the volatile web/network logic from the non-volatile SPIFFS storage mechanics. The UML Class Diagram below illustrates the composition of the main controller, its inheritance of peripheral hardware, and its data aggregation methods:

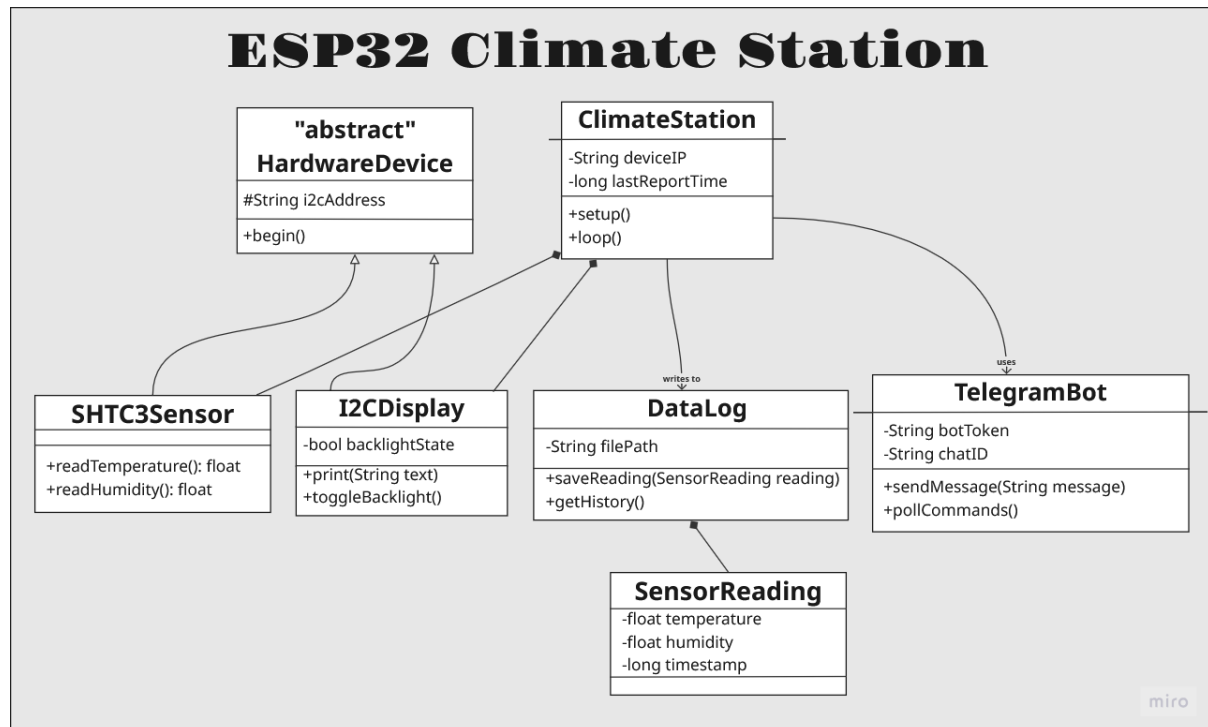


Figure 2: UML Class Diagram illustrating the object-oriented architecture, hardware composition, and data aggregation within the ESP32 firmware.

2.3. User Classes and Characteristics

The primary user class is the "Home User." This user is assumed to have basic familiarity with connecting devices to home Wi-Fi networks and using the Telegram messaging application, but requires no advanced programming or engineering knowledge to operate the deployed system.

2.4. Operating Environment

The firmware executes directly on the ESP32 microcontroller utilizing the C++/FreeRTOS framework. The hardware relies on a standard 2.4GHz 802.11 b/g/n local Wi-Fi network and requires a stable outbound internet connection to communicate with the Telegram API.

2.5. Design and Implementation Constraints

The system is strictly constrained by the ESP32's limited SRAM and SPIFFS flash partition size. Furthermore, to support Over-The-Air (OTA) updates, the compiled firmware binary must never exceed 50% of the allocated application flash space.

2.6. User Documentation

The system will be delivered with a digital `README.md` file containing a quick-start guide, physical wiring schematics for the I2C peripherals, and a reference table of supported Telegram commands.

2.7. Assumptions and Dependencies

The system assumes a stable, continuous 5V power supply via the ESP32's onboard USB port. It depends heavily on the continued availability of the external Telegram Bot API and local Multicast DNS (mDNS) support on the user's home network router.

3. System Features

3.1. Local Climate Monitoring and Display

The system continuously reads ambient temperature and humidity from the I2C sensor and displays the live data on a hardware LCD screen, which can be toggled via a physical button.

3.2. Telegram Bot Integration

The device communicates with the Telegram API to send automated 15-minute climate reports and listens for incoming user commands to control the device remotely.

3.3. Web Dashboard and Device Management

The system hosts a local mDNS web server displaying a Highcharts visualization of the 24-hour data history stored in the SPIFFS memory.

3.4. Active Threshold Alerts

The system actively monitors sensor data against user-defined limits and pushes immediate emergency notifications via Telegram if the climate becomes dangerous.

3.5. Dynamic Web Configuration

The web dashboard provides a settings interface allowing the user to update WiFi credentials, Telegram API tokens, and reporting intervals without flashing new firmware.

4. External Interface Requirements

4.1. User Interfaces Overview

The ESP32 Room Climate Station incorporates three separate user interfaces designed for various interaction scenarios. The major physical interface is a 16x2 I2C LCD panel that shows real-time temperature and humidity information, as well as a hardware push-button that allows you to manually control the screen's lighting. The system uses a Telegram Bot interface for remote interaction, allowing users to manage the device and get structured, periodic climatic data by sending specified textual instructions like `/status` or `/restart`. Finally, a local web dashboard may be accessed via a browser at `climate.Local` provides a graphical user interface. This dashboard uses Highcharts to visualise 24-hour historical data patterns and includes a secure settings panel for dynamically configuring network and API credentials.

4.2. Hardware Interfaces

The software directly interfaces with several specific hardware components connected to the ESP32 microcontroller. The system communicates with both the SHTC3 environmental sensor and the 16x2 LCD module utilizing the Inter-Integrated Circuit (I2C) communication protocol, which requires the software to manage the clock timing and specific device addresses. Additionally, the software monitors a digital input on GPIO pin 4, utilizing hardware debouncing techniques to detect state changes from a physical momentary push-button that triggers the LCD backlight control logic. The system assumes a continuous, stable power supply provided via the ESP32's onboard USB port, as deep-sleep battery optimizations are excluded from this architecture.

4.3. Software Interfaces

To complete its feature set, the system interacts with a number of external and internal software components. Externally, it needs a secure outbound HTTPS connection to communicate with the Telegram Bot API, which is required for both sending outward alert messages and polling the server for incoming user instructions. Internally, the software significantly relies on the ESP32's SPIFFS (Serial Peripheral Interface Flash File System) library to

constantly read and write the rolling 24-hour environmental data buffer to the device's non-volatile flash memory. Furthermore, the firmware runs an asynchronous local web server on standard HTTP port 80 and uses the Multicast DNS (mDNS) protocol to broadcast its local hostname to the network router, guaranteeing that the dashboard is always available without the need for static IP setups.

5. System Features/Modules

5.1. Local Climate Monitoring and Display

The system continuously reads ambient temperature and humidity from the I2C sensor and displays the live data on a hardware LCD screen, which can be toggled via a physical button.

- **REQ-1.1:** The system shall measure ambient temperature and humidity using the connected SHTC3 sensor.
- **REQ-1.2:** The system shall display the current temperature and humidity readings locally on the 16x2 I2C LCD.
- **REQ-1.3:** The system shall allow the user to manually toggle the LCD backlight state (on/off) using a physical hardware button connected to GPIO 4.

5.2. Telegram Bot Integration

The device communicates with the Telegram API to send automated 15-minute climate reports and listens for incoming user commands to control the device remotely.

The following Sequence Diagram illustrates the dynamic chronological interaction between the user, the cloud API, and the local hardware when a remote command is executed:

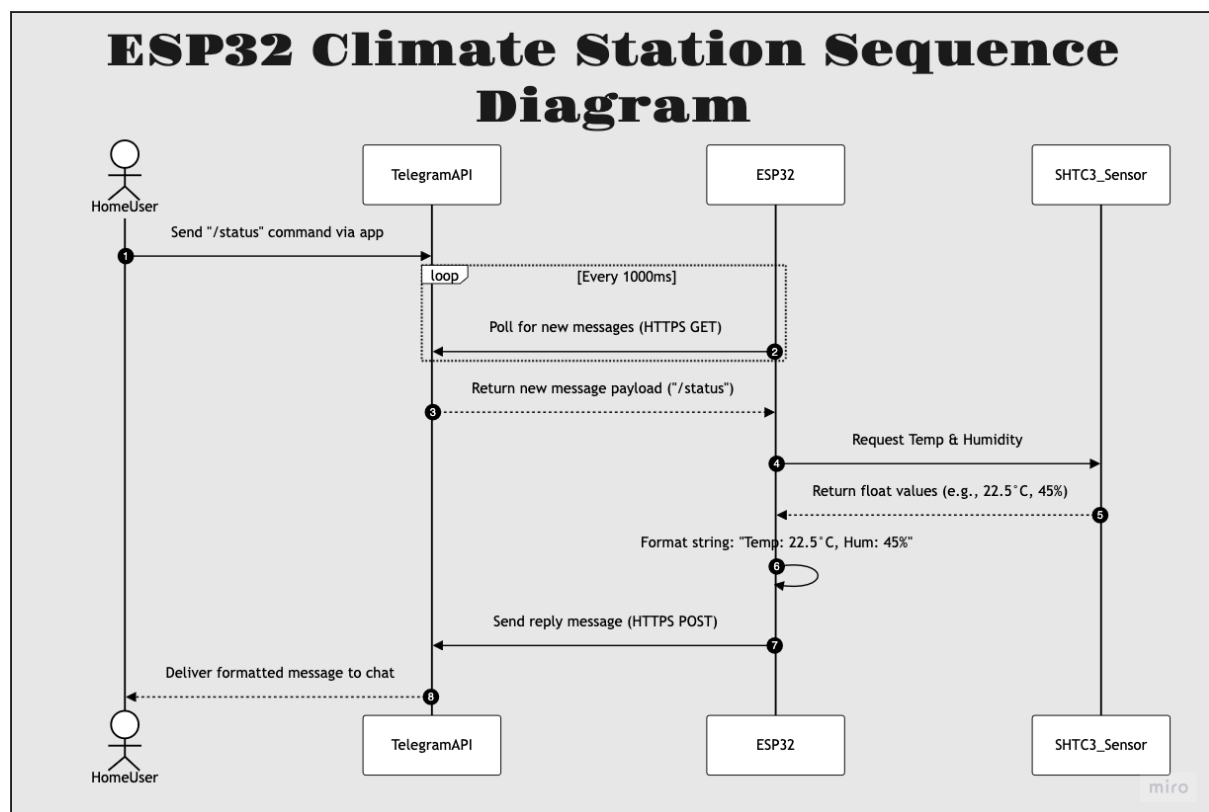


Figure 3: Sequence Diagram detailing the network polling loop and hardware execution of a Telegram /status command.

- **REQ-2.1:** The system shall automatically transmit a formatted ambient climate report to the configured Telegram Bot every 15 minutes.
- **REQ-2.2:** The system shall parse and execute the following remote commands received via Telegram: `/light_on` (enables LCD), `/light_off` (disables LCD), `/status` (returns current readings), and `/restart` (initiates a soft reboot).

- **REQ-2.3:** The system shall clear the Telegram message queue immediately upon booting to prevent infinite restart loops triggered by stale commands.

5.3. Web Dashboard and Device Management

The system hosts a local mDNS web server displaying a Highcharts visualization of the 24-hour data history stored in the SPIFFS memory.

- **REQ-3.1:** The system shall serve a local web dashboard accessible via `http://climate.local` that displays a Highcharts-powered visualization of both live trends and 24-hour historical data.

5.4. Active Threshold Alerts

The system actively monitors sensor data against user-defined limits and pushes immediate emergency notifications via Telegram if the climate becomes dangerous.

- **REQ-4.1:** The system shall store user-defined maximum and minimum threshold values for both temperature and humidity in non-volatile memory.
- **REQ-4.2:** The system shall immediately transmit an emergency alert message via the Telegram Bot if a polled sensor reading falls outside the defined threshold limits.

5.5. Dynamic Web Configuration

The web dashboard provides a settings interface allowing the user to update WiFi credentials, Telegram API tokens, and reporting intervals without flashing new firmware.

- **REQ-5.1:** The system shall provide a secure settings route on the web dashboard (`http://climate.local/settings`).
- **REQ-5.2:** The system shall allow the user to input and save new WiFi credentials, Telegram Bot tokens, and Telegram Chat IDs via the web interface.

6. Nonfunctional Requirements

6.1. Performance

- **NF-1.1:** The system shall utilize non-blocking timers for network operations to ensure that the physical hardware toggle (GPIO 4) and local LCD updates remain responsive, acting with less than a 500-millisecond delay.

6.2. Reliability

- **NF-2.1:** The system shall persist a rolling 24-hour history of sensor data to internal SPIFFS flash memory to ensure data is preserved and reloaded across unexpected device restarts.

The following State Diagram illustrates the firmware lifecycle, specifically highlighting how the system gracefully handles network timeouts by entering an Offline Mode and continually attempting to recover the connection without halting local monitoring:

ESP32 Climate Station State Diagram

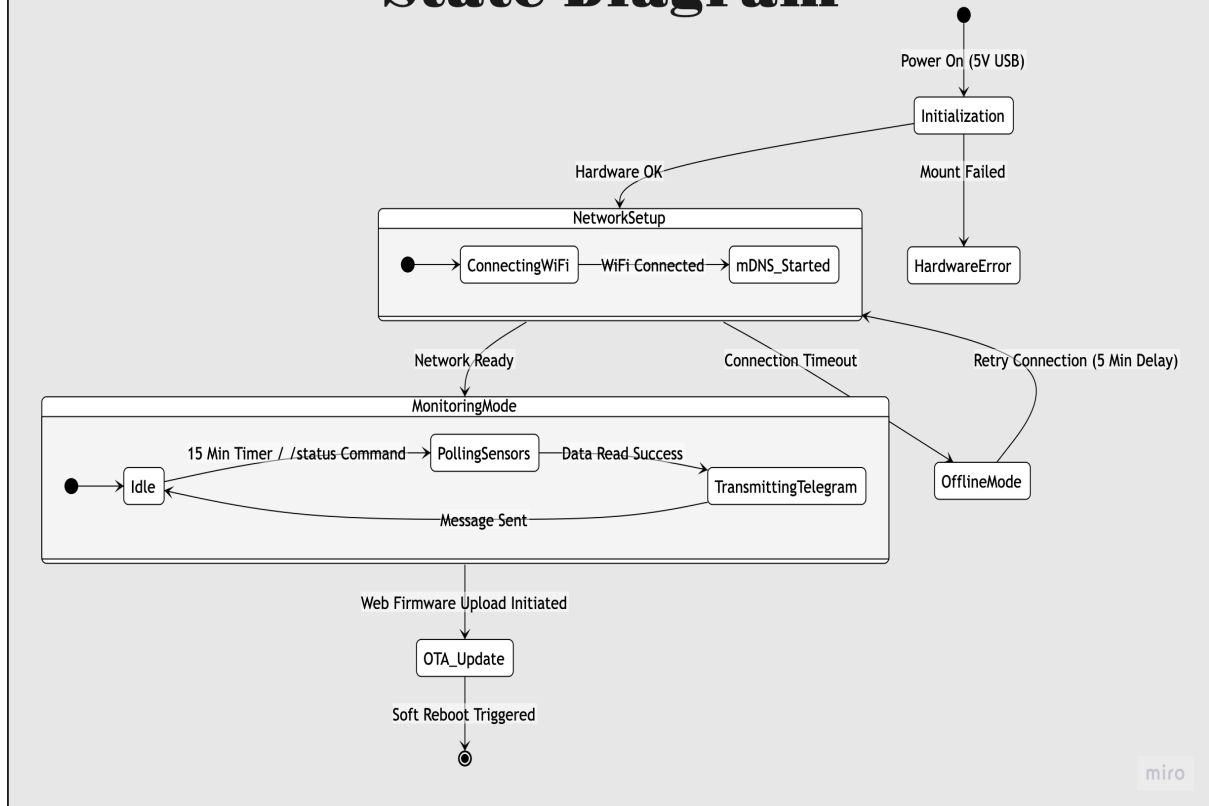


Figure 4: State Diagram modelling the ESP32 firmware lifecycle, network recovery, and OTA transition.

To further support this reliability constraint, the firmware manages the internal flash memory using a rolling buffer mechanism. The following Activity Diagram illustrates the data logging algorithm, demonstrating how the system continuously frees up memory to prevent a catastrophic SPIFFS overflow crash:

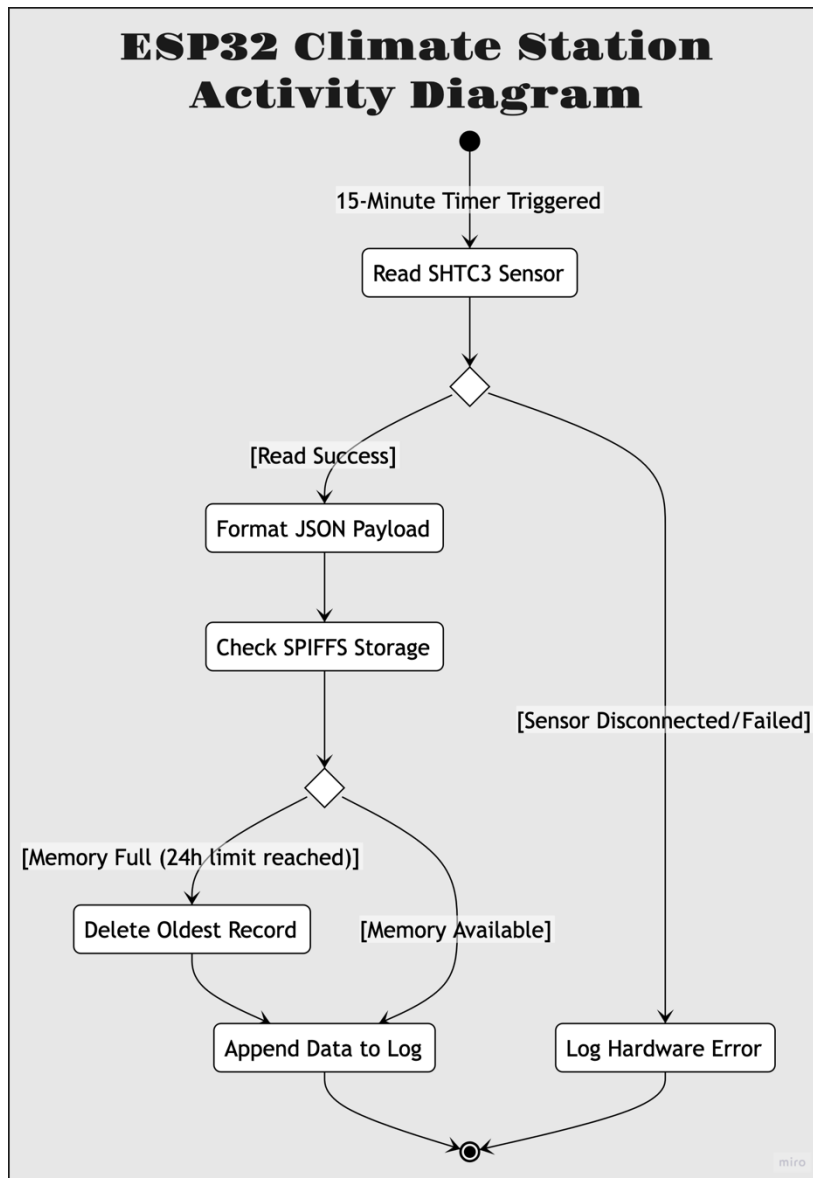


Figure 5: ESP32 Climate Station Activity Diagram

6.3. Maintainability

- **NF-3.1:** The system shall support Over-The-Air (OTA) firmware updates via a local web browser interface, allowing deployment of new code without a physical USB connection.

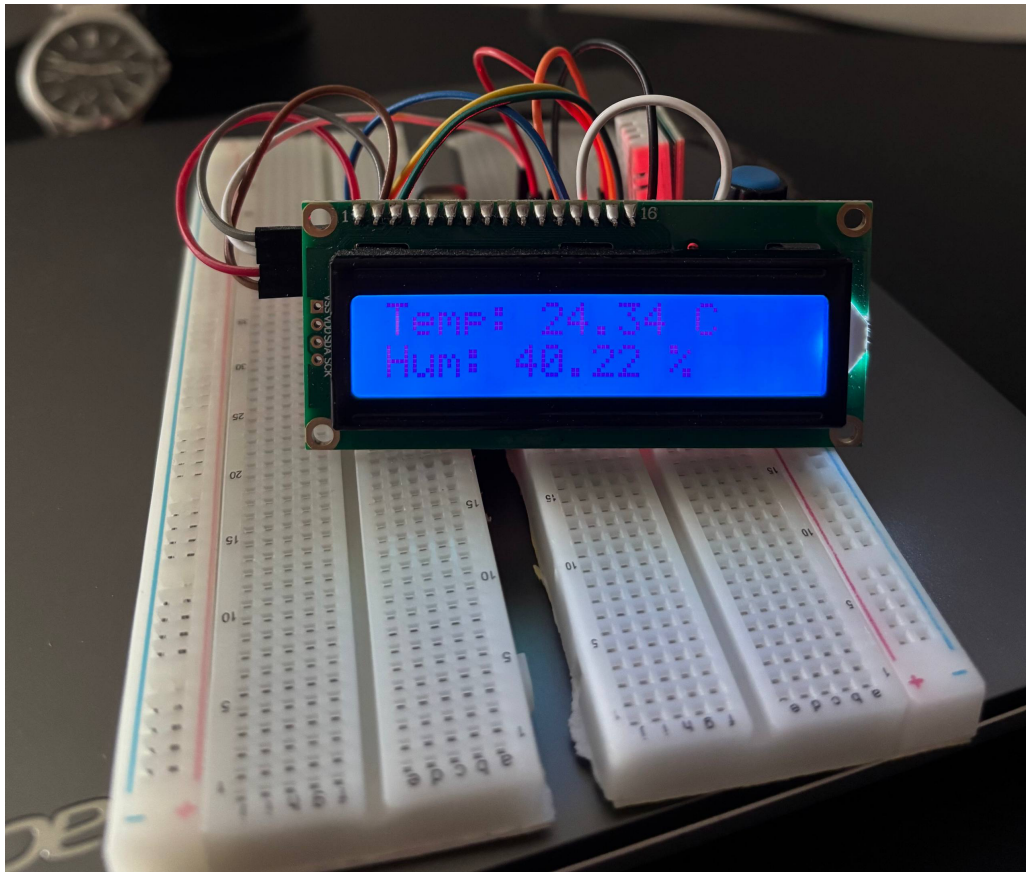


Figure 6: Photo of the Physical Module presented for the final project defense.

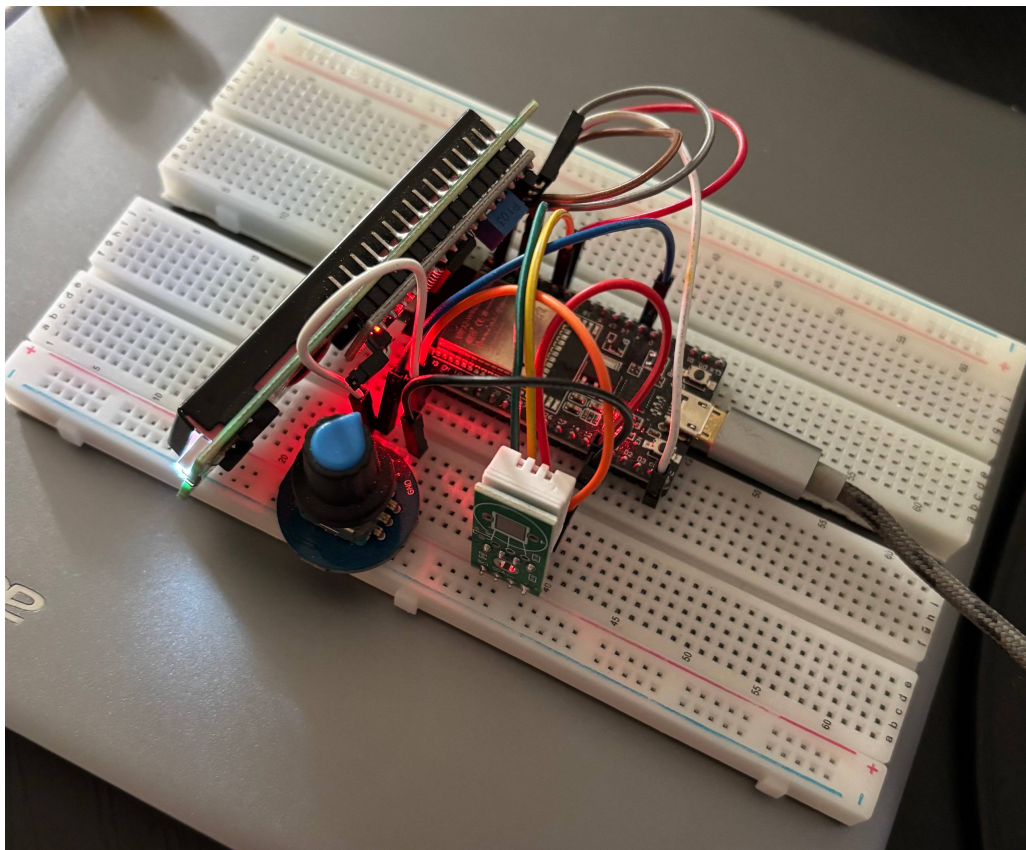


Figure 7: Photo of the Physical Module from above.